

Assignment – 15.

1. Create a Redux slice named `playlistSlice` to manage a Spotify-style playlist with an array of song objects (each with `id`, `title`, and `artist`). Use `useSelector` to display the list of songs in a `Playlist.js` component.
2. Add a feature in your `Playlist.js` component to let users add a new song by dispatching an `addSong` action using `useDispatch`. The new song should be appended to the Redux state and displayed in the playlist.

Hint: Use a simple form with title and artist fields, and generate a unique id for each new song.
3. Install and configure Redux DevTools in your React app. Perform actions like adding or removing songs from the playlist and observe the state changes in Redux DevTools. Take a screenshot of the DevTools showing at least one state update.
4. Given a React app that uses Context API to manage a shopping cart (`cartContext.js`), refactor the state management to use Redux instead. Briefly describe one advantage and one disadvantage you noticed when switching from Context to Redux for this use case.

Answer:- 1. Install Redux Toolkit

`npm install @reduxjs/toolkit react-redux`

`playlistSlice.js`

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  songs: [
    {
      id: 1,
      title: "Shape of You",
      artist: "Ed Sheeran",
    },
  ],
};

const playlistSlice = createSlice({
  name: "playlist",
  initialState,
  reducers: {
    addSong: (state, action) => {
```

```

        state.songs.push(action.payload);
    },

    removeSong: (state, action) => {
        state.songs = state.songs.filter(
            (song) => song.id !== action.payload
        );
    },
},
});

export const { addSong, removeSong } =
    playlistSlice.actions;

export default playlistSlice.reducer;

```

store.js

```

import { configureStore } from "@reduxjs/toolkit";
import playlistReducer from "../playlistSlice";

export const store = configureStore({
    reducer: {
        playlist: playlistReducer,
    },
});

```

Main.js

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "../App";

import { Provider } from "react-redux";
import { store } from "../redux/store";

ReactDOM.createRoot(
    document.getElementById("root")
).render(
    <Provider store={store}>
        <App />
    </Provider>
);

```

Playlist.js

```

import { useState } from "react";
import {
  useDispatch,
  useSelector,
} from "react-redux";

import {
  addSong,
  removeSong,
} from "../redux/playlistSlice";

const Playlist = () => {
  const dispatch = useDispatch();

  const songs = useSelector(
    (state) => state.playlist.songs
  );

  const [title, setTitle] = useState("");
  const [artist, setArtist] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();

    if (!title || !artist) return;

    dispatch(
      addSong({
        id: Date.now(),
        title,
        artist,
      })
    );

    setTitle("");
    setArtist("");
  };

  return (
    <div style={{ padding: "20px" }}>
      <h1>Spotify Playlist</h1>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Song Title"
          value={title}
          onChange={(e) =>

```

```

        setTitle(e.target.value)
    }
  />

  <br />
  <br />

  <input
    type="text"
    placeholder="Artist"
    value={artist}
    onChange={(e) =>
      setArtist(e.target.value)
    }
  />

  <br />
  <br />

  <button type="submit">
    Add Song
  </button>
</form>

<hr />

<h2>Playlist</h2>

{songs.map((song) => (
  <div key={song.id}>
    <strong>{song.title}</strong>
    { " - " }
    {song.artist}

    <button
      onClick={() =>
        dispatch(removeSong(song.id))
      }
      style={{ marginLeft: "10px" }}
    >
      Remove
    </button>
  </div>
))}
</div>
);
};

```

```
export default Playlist;
```

App.jsx

```
import Playlist from "../components/Playlist";

function App() {
  return <Playlist />;
}

export default App;
```